Assignment 4: Assignments on In-built Data Structures

Name: Rajesh Mahendra Anerao

Sub: Python for Engineers (PFE)

Roll No.: 15

Branch: CSE-Data Science Engineering

Q 1.

Problem Statement: Program to Create and view elements of a list

9.py

D: > Desktop > Assignment & Notes > Python > 9.py > ...

```python
list=[11,22,45,96,78,54,32,12,9]

print("list of elements are:",list)
for i in list:
  print(i)
```

Output:

```
PS C:\Users\Lenovo> & C:/Users/Lenovo/AppData/Local/Programs/Python/Python311
list of elements are: [11, 22, 45, 96, 78, 54, 32, 12, 9]
11
22
45
96
78
54
32
12
9
PS C:\Users\Lenovo>
```

Conclusions:

Duplicate items can be removed from a list using set().

Q 2.

Problem Statement: Program to Create and view elements of a tuple

```
10.py   9.py
10.py > ...
tuple=(11,22,45,96,78,54,32,12,9)

print("tuple of elements are:",tuple)
for i in tuple:
  print(i)
```

Output:

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS D:\Desktop\Assignment & Notes\Python> & C:/Users/U
thon.exe "d:/Desktop/Assignment & Notes/Python/10.py"
11
22
45
96
78
54
32
12
9
PS D:\Desktop\Assignment & Notes\Python>
```

Conclusions:

Duplicate items can be removed from a list using set().

Q 3.

Problem Statement: Program to access List Index and Values

```
11.py
11.py > ...
list=[12,15,18,21,24,27,30]

for i in range(len(list)):
    print("Index:", i, "Value:", list[i])
```

Output:

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL

PS D:\Desktop\Assignment & Notes\Python> & C
thon/11.py"
Index: 0 Value: 12
Index: 1 Value: 15
Index: 2 Value: 18
Index: 3 Value: 21
Index: 4 Value: 24
Index: 5 Value: 27
Index: 6 Value: 30
PS D:\Desktop\Assignment & Notes\Python>
```

Conclusion:

A set stores only unique elements.

Q 4.

Problem Statement: Program to add two Lists

```
12.py
12.py > ...
1  list1=[10,20,30,40,50]
2  list2=[60,70,80,90,100]
3
4  list3=list1+list2
5  print(list3)
```

Output:

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS D:\Desktop\Assignment & Notes\Python> & C:/Users/
thon/12.py"
[10, 20, 30, 40, 50, 60, 70, 80, 90, 100]
PS D:\Desktop\Assignment & Notes\Python>
```

Conclusion:

Dictionary values can be viewed using .values().

Q 5.

Problem Statement: Program to check if a List is Empty or Not

```python
list=[10,12,14,16,18,20]

if len(list)==0:
  print("List is empty")
else:
  print("List is not empty")
```

Output:

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL

PS D:\Desktop\Assignment & Notes\Python> & C
List is not empty
PS D:\Desktop\Assignment & Notes\Python>
```

Conclusion:

Dictionary keys can be viewed using .keys().

Q 6.

Problem Statement: Program to Find the Largest Number in a List

```python
list1=[1,2,3,4,5]

largest=list1[0]
for i in list1:
    if i>largest:
        largest=i
print("largest number is",largest)
```

Output:

```
PS D:\Desktop\Assignment & Notes\Python> & C:/
thon/14.py"
largest number is 5
PS D:\Desktop\Assignment & Notes\Python>
```

Conclusion:

New items can be inserted and existing items can be deleted from a dictionary.

Q 7.

Problem Statement: Program to Find the Second Largest Number in a List

```python
list=[22,23,45,64,78,81]

list.sort()

print("Second Largest number is: ", list[-2])
```

Output:

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    P

PS D:\Desktop\Assignment & Notes\Python> & C:/U
thon/15.py"
Second Largest number is:  78
PS D:\Desktop\Assignment & Notes\Python>
```

Conclusion:

A dictionary can be sorted by values in ascending or descending order.

Q 8.

Problem Statement: Program to Put Even and Odd elements in a List into Two Different Lists

```
16.py    15.py
16.py > ...
list=[1,4,7,10]

even=[]
odd=[]
for i in list:
    if i%2==0:
        even.append(i)
    else:
        odd.append(i)
print("Even numbers are: ", even)
print("Odd numbers are: ", odd)
```

Output:

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS D:\Desktop\Assignment & Notes\Python> & C:/User
thon/16.py"
Even numbers are:  [4, 10]
Odd numbers are:  [1, 7]
PS D:\Desktop\Assignment & Notes\Python>
```

Conclusion:

Two dictionaries can be combined to create a new dictionary.

Q 9.

Problem Statement: Program to Find all Numbers in a Range which are Perfect Squares and Sum of all Digits in the Number is Less than 10

```python
start = int(input("Enter start: "))
end = int(input("Enter end: "))

result = [num for num in range(start, end + 1)
                if int(num ** 0.5) ** 2 == num and sum(map(int, str(num))) < 10]

print(result)
```

Output:

```
PS D:\Desktop\Assignment & Notes\Python> & C
thon/17.py"
Enter start: 2
Enter end: 8
[4]
PS D:\Desktop\Assignment & Notes\Python>
```

Conclusion:

The in operator checks whether a key exists in a dictionary.

Q 10.

Problem Statement: Program to Generate Random Numbers from 1 to 20 and Append Them to the List

```python
import random

list=[]
for i in range(10):
    list.append(random.randint(1,100))
print(list)
```

Output:

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL

PS D:\Desktop\Assignment & Notes\Python> & c
thon/18.py"
[80, 98, 71, 36, 2, 59, 37, 43, 36, 34]
PS D:\Desktop\Assignment & Notes\Python>
```

Conclusion:

Two dictionaries can be merged into one.

Q 11.

Problem Statement: Program to Remove the Duplicate Items from a List

```python
list=[1,2,2,3,3,4,4,4,5]

list1=[]
for i in list:
  if i not in list1:
    list1.append(i)
print(list1)
```

Output:

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMIN

PS D:\Desktop\Assignment & Notes\Python>
thon/19.py"
[1, 2, 3, 4, 5]
PS D:\Desktop\Assignment & Notes\Python>
```

Conclusion:

Maximum and minimum values in a dictionary can be found using max() and min().

Q 12.

Problem Statement: Program to create and view a dictionary

```
20.py
20.py > ...
1  student = {
2      "name": "Raj",
3      "age": 18,
4      "course": "Python"
5  }
6
7  print(student)
```

Output:

```
PS D:\Desktop\Assignment & Notes\Python> & C:/Us
thon/20.py"
{'name': 'Raj', 'age': 18, 'course': 'Python'}
PS D:\Desktop\Assignment & Notes\Python>
```

Conclusion:

Multiple elements from a list can be added to a set using .update().

Q 13.

Problem Statement: Program to create and view elements of a set

```
21.py
21.py > ...
1  set = {10, 20, 30, 40}
2
3  print("Set elements:", set)
```

Output:

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL

PS D:\Desktop\Assignment & Notes\Python> & C:
thon/21.py"
Set elements: {40, 10, 20, 30}
PS D:\Desktop\Assignment & Notes\Python>
```

Conclusion:

symmetric_difference_update() keeps only elements not common in both sets.

Q 14.

Problem Statement: Program to Print values of dictionary

```python
student = {
    "name": "Raj",
    "age": 18,
    "course": "Python"
}

print("Dictionary Values:")
for value in student.values():
    print(value)
```

Output:

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL

PS D:\Desktop\Assignment & Notes\Python> & C:
thon/22.py"
Dictionary Values:
Raj
18
Python
PS D:\Desktop\Assignment & Notes\Python>
```

Conclusion:

symmetric_difference() returns elements present in either set but not in both.

Q 15.

Problem Statements: Program to print all keys of dictionary

```python
student = {
    "name": "Raj",
    "age": 18,
    "course": "Python"
}

print("Dictionary Keys:")
for key in student.keys():
    print(key)
```

Output:

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL

PS D:\Desktop\Assignment & Notes\Python> & 
thon/23.py"
Dictionary Keys:
name
age
course
PS D:\Desktop\Assignment & Notes\Python>
```

Conclusion:

isdisjoint() checks whether two sets have common elements.

Q 16.

Problem Statements: Program to insert and delete from dictionary

```python
student = {
    "name": "Raj",
    "age": 18
}

student["course"] = "Python"
print("After Insertion:", student)

del student["age"]
print("After Deletion:", student)
```

Output:

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS D:\Desktop\Assignment & Notes\Python> & C:/Users/Lenovo/AppDa
thon/24.py"
After Insertion: {'name': 'Raj', 'age': 18, 'course': 'Python'}
After Deletion: {'name': 'Raj', 'course': 'Python'}
PS D:\Desktop\Assignment & Notes\Python>
```

Conclusion:

intersection_update() keeps only the common elements in both sets**.**

Q 17.

Problem Statement: Program to sort (ascending and descending) a dictionary by value

```python
marks = {
    "Math": 85,
    "Science": 92,
    "English": 78
}

ascending = dict(sorted(marks.items(), key=lambda item: item[1]))
descending = dict(sorted(marks.items(), key=lambda item: item[1], reverse=True))

print("Ascending Order:", ascending)
print("Descending Order:", descending)
```

Output:

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS D:\Desktop\Assignment & Notes\Python> & C:/Users/Lenovo/AppD
thon/25.py"
Ascending Order: {'English': 78, 'Math': 85, 'Science': 92}
Descending Order: {'Science': 92, 'Math': 85, 'English': 78}
PS D:\Desktop\Assignment & Notes\Python>
```

Conclusion:

A list can be created and displayed using square brackets [].

Q 18.

Problem Statement: Program to concatenate dictionaries to create a new one

```python
26.py > ...
dict1 = {1: "A", 2: "B"}
dict2 = {3: "C", 4: "D"}

new_dict = {**dict1, **dict2}

print("Concatenated Dictionary:", new_dict)
```

Output:

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS D:\Desktop\Assignment & Notes\Python> & C:/Users/Lenovo/
thon/26.py"
Concatenated Dictionary: {1: 'A', 2: 'B', 3: 'C', 4: 'D'}
PS D:\Desktop\Assignment & Notes\Python>
```

Conclusion:

A tuple can be created and displayed using parentheses ().

Q 19.

Problem Statement: Program to check whether a given key already exists in a dictionary.

```python
student = {
    "name": "Raj",
    "age": 18
}

key = "name"

if key in student:
    print("Key exists")
else:
    print("Key does not exist")
```

Output:

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL

PS D:\Desktop\Assignment & Notes\Python> & C:/
thon/27.py"
Key exists
PS D:\Desktop\Assignment & Notes\Python>
```

Conclusion:

List items can be accessed using their index numbers

Q 20.

Problem Statement: Program to merge two Python dictionaries

```python
dict1 = {"a": 1, "b": 2}
dict2 = {"c": 3, "d": 4}

merged_dict = dict1 | dict2

print("Merged Dictionary:", merged_dict)
```

Output:

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS D:\Desktop\Assignment & Notes\Python> & C:/Users/Ler
thon/28.py"
Merged Dictionary: {'a': 1, 'b': 2, 'c': 3, 'd': 4}
PS D:\Desktop\Assignment & Notes\Python>
```

Conclusion:

Two lists can be added using the + operator

Q 21.

Problem Statement: Program to get the maximum and minimum value in a dictionary

```python
marks = {
    "Math": 85,
    "Science": 92,
    "English": 78
}

max_value = max(marks.values())
min_value = min(marks.values())

print("Maximum Value:", max_value)
print("Minimum Value:", min_value)
```

Output:

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS D:\Desktop\Assignment & Notes\Python> & C:/Users/
thon/29.py"
Maximum Value: 92
Minimum Value: 78
PS D:\Desktop\Assignment & Notes\Python>
```

Conclusion:

A list is empty if its length is 0.

Q 22.

Problem Statement: Program to Add a list of elements to a set

```python
set1 = {10, 20, 30}
set2 = {20, 40, 50}

set1.symmetric_difference_update(set2)

print("Updated Set1:", set1)
```

Output:

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    P

PS D:\Desktop\Assignment & Notes\Python> & C:/U
thon/29.py"
Updated Set1: {40, 10, 50, 30}
PS D:\Desktop\Assignment & Notes\Python>
```

Conclusion:

The largest number in a list can be found using max()

Q 23.

Problem Statement: Program to Update the first set with items that don't exist in the second set

```python
A = {1, 2, 3, 4}
B = {3, 4, 5, 6}

result = A.symmetric_difference(B)

print("Elements present in A or B but not both:", result)
```

Output:

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS D:\Desktop\Assignment & Notes\Python> & C:/Users/Le
thon/30.py"
Elements present in A or B but not both: {1, 2, 5, 6}
PS D:\Desktop\Assignment & Notes\Python>
```

Conclusion:

The second largest number can be found by sorting the list

Q 24.

Problem Statement: Program to Check if two sets have any elements in common.

```
31.py
31.py > ...
set1 = {1, 2, 3}
set2 = {3, 4, 5}

if set1.isdisjoint(set2):
    print("No common elements")
else:
    print("Sets have common elements")
```

Output:

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL

PS D:\Desktop\Assignment & Notes\Python> & C:
thon/31.py"
Sets have common elements
PS D:\Desktop\Assignment & Notes\Python>
```

Conclusion:

Even and odd numbers can be separated into different lists

Q 25.

Problem Statement: Program to Remove items from set1 that are not common to both set1 and set2

```python
set1 = {10, 20, 30, 40}
set2 = {20, 30, 50, 60}

set1.intersection_update(set2)

print("Updated set1:", set1)
```

Output:

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL

PS D:\Desktop\Assignment & Notes\Python> & C:
thon/32.py"
Updated set1: {20, 30}
PS D:\Desktop\Assignment & Notes\Python>
```

Conclusion:

Perfect squares with digit sum less than 10 can be found using loops and conditions.

Q 26.

Problem Statement: Program to Return a set of elements present in Set A or B, but not both

```python
A = {1, 2, 3, 4}
B = {3, 4, 5, 6}

result = A.symmetric_difference(B)

print("Elements present in A or B but not both:", result)
```

Output:

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS D:\Desktop\Assignment & Notes\Python> & C:/Users/Lenc
Elements present in A or B but not both: {1, 2, 5, 6}
PS D:\Desktop\Assignment & Notes\Python>
```

Conclusion:

Perfect squares with digit sum less than 10 can be found using loops and conditions.